
Durin's Code

Team Reflection Report

CS4031 — Compiler Construction
Spring 2026

Student Name	Student ID
Huzaifa Abdul Rehman	23K-0782
Abdul Moiz Hussain	23K-0553
Muhammad Abdullah Khan	23K-0607

Contents

1	Project Summary	2
2	Division of Work	2
3	What Went Well	2
3.1	Clear Phase Boundaries	2
3.2	Memory Safety via <code>unique_ptr</code>	2
3.3	Context-Sensitive Lexing for the Dot Operator	3
3.4	TAC as a Stable Interface	3
3.5	Test-Driven Discovery of Bugs	3
4	Challenges We Faced	3
4.1	Parser Priming Bug	3
4.2	<code>unique_ptr</code> Copy vs. Move	3
4.3	Lexer Line and Column Tracking	3
4.4	Two-Pass Semantic Analysis Ordering	4
4.5	Condition Lowering to TAC	4
5	What We Would Do Differently	4
5.1	Integration Tests Between Adjacent Phases	4
5.2	Richer Static Type System	4
5.3	Improved REPL Experience	4
5.4	Error-Tolerant Parser	4
6	Lessons Learned	5
7	Conclusion	5

1 Project Summary

Durin's Code is a complete compiler for a Tolkien-themed text-adventure domain-specific language, built from scratch in C++17 as the CS4031 course project for Spring 2026. We implemented the full seven-phase compilation pipeline: a hand-written lexer, a recursive-descent parser producing a typed AST, a two-pass semantic analyser with a global symbol table, a Three-Address Code (TAC) intermediate representation generator, a two-pass optimiser (dead-code elimination and redundant-AND folding), a JSON bytecode serialiser, and a virtual machine with an interactive game loop.

The final system compiles and executes `.dc` programs entirely from the terminal with no external runtime dependencies beyond the vendored `nlohmann/json` v3.11.3 library. The test suite comprises 49 individual unit tests across six GoogleTest suites, all of which pass.

2 Division of Work

Phase / Deliverable	Primary Owner	Student ID
Lexer (<code>lexer.cpp</code>) + lexer tests	Abdul Moiz Hussain	23K-0553
Parser (<code>parser.cpp</code>) + parser tests	Abdul Moiz Hussain	23K-0553
Semantic Analyser + semantic tests	Muhammad Abdullah Khan	23K-0607
TAC Generator + Optimiser + TAC tests	Muhammad Abdullah Khan	23K-0607
Code Generator + codegen tests	Huzaifa Abdul Rehman	23K-0782
Virtual Machine + VM tests	Huzaifa Abdul Rehman	23K-0782
CLI (<code>main.cpp</code>)	Huzaifa Abdul Rehman	23K-0782
Language Reference (LaTeX)	Muhammad Abdullah Khan	23K-0607
Architecture Document (LaTeX)	Huzaifa Abdul Rehman	23K-0782
Example programs (<code>examples/</code>)	All three members	—
Integration testing + final smoke tests	All three members	—

3 What Went Well

3.1 Clear Phase Boundaries

Organising the project as one CMake static library per phase (`durins_lexer`, `durins_parser`, and so on) meant each team member could own a phase independently. Changes in one phase rarely broke another because the only shared surface was the typed header interface. GoogleTest suites gave immediate feedback whenever an interface changed.

3.2 Memory Safety via `unique_ptr`

Using `unique_ptr` throughout the AST eliminated entire classes of memory bugs from the start. Ownership semantics were strict — every child node had exactly one owner — which paid dividends when we later added move semantics to `push_back` calls. There were no use-after-free or double-free issues in the parser or downstream passes.

3.3 Context-Sensitive Lexing for the Dot Operator

Rather than adding parser lookahead to distinguish `player.has_item(...)` from a generic attribute access, we encoded the distinction directly in the lexer by tracking `lastEmitted` and `lastBeforeDot`. This kept the parser simpler and produced a richer, more semantically meaningful token stream.

3.4 TAC as a Stable Interface

Once the 18-opcode instruction set was agreed upon, both the TAC generator and the VM could be developed independently against that contract. The JSON bytecode format doubled as a clean separation boundary *and* gave us the `-o` bytecode-export feature essentially for free.

3.5 Test-Driven Discovery of Bugs

Writing test suites before or alongside each phase surfaced several non-obvious bugs early — most notably the parser priming bug, the `unique_ptr` copy issue, and the double-line-count in the lexer. All 49 tests now pass.

4 Challenges We Faced

4.1 Parser Priming Bug

The original parser never called `advance()` after `initLexer()`, leaving `parser.current` zero-initialised. Every compilation attempt failed with “Expect 'room' or 'action' at line 0 col 0” before reading a single token. The fix was a single `advance()`; call at the top of `parseProgram()`, but diagnosing it required careful comparison of the parser's internal model against the actual token stream state.

4.2 `unique_ptr` Copy vs. Move

Several `push_back(ptr)` calls on `vector<unique_ptr<T>>` compiled silently under GCC but were correctly rejected by Clang as illegal copies. Systematically replacing every `push_back(ptr)` with `push_back(std::move(ptr))` resolved the issue and surfaced the correct ownership semantics. This bug taught us that relying on one compiler's leniency can hide portability problems.

4.3 Lexer Line and Column Tracking

The original `advance()` function incremented `lexer.line` on `\n`, but `skipWhiteSpace()` also incremented it for the same newline character, causing double-counting. Windows `\r\n` endings additionally counted as two separate newlines. Fixing both required establishing a single rule: `advance()` is the *only* function that may modify `lexer.line` and `lexer.column`.

4.4 Two-Pass Semantic Analysis Ordering

Exits reference rooms that may be declared later in the file. A naive single-pass approach incorrectly reported “room not found” for all forward references. Restructuring into two passes — register everything first, validate second — resolved this completely but required careful separation of registration and validation logic into distinct recursive walks of the AST.

4.5 Condition Lowering to TAC

Generating correct TAC for compound conditions (`&&`, `||`, nested `has_item`) required careful management of temporary boolean registers and consistent label numbering for `JUMP_IF_FALSE` / `JUMP` pairs. Getting label numbering correct for nested `if` statements — particularly when no `else` branch is present — required several iterations of the `emitCondition()` function.

5 What We Would Do Differently

5.1 Integration Tests Between Adjacent Phases

We wrote per-phase unit test suites but relied on full end-to-end runs to test the integration between adjacent phases (e.g., whether the semantic analyser correctly feeds type information to the TAC generator). Dedicated integration tests at each phase boundary would have caught interface mismatches earlier and more precisely.

5.2 Richer Static Type System

The current type system is implicit: property types are inferred from their literal values at parse time and not enforced in subsequent phases. A formal type environment threaded through the pipeline would enable better error messages for invalid assignments such as `player.health = "text"` and would lay the groundwork for type-driven optimisations.

5.3 Improved REPL Experience

The `--interactive` REPL compiles a complete snippet on double-Enter. A proper incremental REPL would support persistent room and action definitions across submissions, allowing iterative world-building from the command line. This would make the tool significantly more useful for rapid prototyping.

5.4 Error-Tolerant Parser

The current parser uses panic-mode recovery, which sometimes discards large portions of the input to find the next synchronisation point. A more sophisticated error-recovery strategy (e.g., insertion/deletion of a single token) would produce better error messages for common mistakes like a missing closing brace.

6 Lessons Learned

Compiler phases are natural unit boundaries. Each phase has a clearly typed input and output. This makes parallel development practical, makes it easy to substitute implementations, and produces a codebase that can be read one phase at a time.

Static validation pays dividends at runtime. Every semantic check we added to the compiler saved us from writing equivalent defensive code in the VM. The VM could be written assuming valid input, making it significantly simpler and more predictable.

Intermediate representations decouple work. The TAC program and JSON byte-code are explicit, inspectable intermediate forms. The `--debug` flag that dumps the symbol table and optimised TAC was invaluable during development. We learned: build your IR to be human-readable from day one.

Test suites are not optional for compiler development. The 49 tests across six suites caught regressions immediately whenever we modified a shared data structure. Without them, cross-phase changes would have required manual end-to-end testing for every modification — a process that scales poorly.

Ownership semantics should be explicit. Using `unique_ptr` made ownership immediately visible in the type signature. By contrast, the raw-pointer approach of the original lexer (which stores a pointer into the source buffer) required careful documentation to be safe. The lesson: prefer types that express ownership invariants over comments that describe them.

7 Conclusion

Building Durin's Code from a proposal to a working compiler with 49 passing tests gave our team practical experience with every stage of the compilation pipeline. The project demonstrated that a relatively small DSL — fewer than 15 keywords and a handful of statement types — nevertheless requires a carefully engineered multi-phase compiler to be robust and maintainable.

The challenges we encountered (parser priming, ownership semantics, two-pass analysis, condition lowering) are the same classes of problems faced in production compiler development. Working through them ourselves, rather than studying them in the abstract, produced a significantly deeper understanding of the material.